Software Asset Management (SAM) Program Standup & Governance

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — Software Asset Management (SAM) Program Standup & Governance 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Software Asset Management (SAM) Program Standup & Governance 5

What a SAM program is . . . 5

Why this matters . . . 5

The SAM lifecycle . . . 5

Roles and RACI . . . 5

Standard operating procedures and workflows . . . 6

KPIs and the system of record . . . 6

ITIL v4 alignment . . . 6

What this is not . . . 6

The deployment model . . . 6

2. Requirements Guide — Software Asset Management (SAM) Program Standup & Governance 6

2.1 A system of record for entitlements . . . 7

2.2 Deployment / usage discovery data . . . 7

2.3 A request / intake channel . . . 7

2.4 Named owners and a RACI . . . 7

2.5 Executive sponsorship . . . 7

2.6 A governance cadence and forum . . . 7

2.7 A pilot scope . . . 8

Checklist . . . 8

3. Implementation Guide — Software Asset Management (SAM) Program Standup & Governance 8

Step 1 — Assess current state . . . 8

Step 2 — Define scope and roles . . . 9

Step 3 — Stand up the system of record . . . 9

Step 4 — Stand up the intake channel . . . 9

Step 5 — Author the SOP set . . . 9

Step 6 — Define the KPI set and reporting . . . 10

Step 7 — Establish the governance cadence . . . 10

Step 8 — Build the pilot scope . . . 10

Step 9 — Run the first governance cycle on the pilot . . . 10
Step 10 — Validate against the pilot gate . . . 11
Step 11 — Expand across the estate . . . 11
5. Testing Guide — Software Asset Management (SAM) Program Standup & Governance 11
Test 1 — System of record is authoritative . . . 11
Test 2 — Intake routes a request . . . 11
Test 3 — Roles are owned . . . 11
Test 4 — Discovery data is available . . . 11
Test 5 — Reconciliation computes an ELP . . . 12
Test 6 — KPI set reports . . . 12
Test 7 — Governance cadence runs . . . 12
Pass/fail gate . . . 12
6. Troubleshooting Guide — Software Asset Management (SAM) Program Standup & Governance 12
A. Reconciliation cannot compute an ELP . . . 12
B. Requests bypass the program . . . 13
C. A lifecycle stage stalls . . . 13
D. KPI report has gaps . . . 13
E. Graph scripts fail to connect . . . 13
F. Program drifts back to spreadsheets . . . 13
G. Diagnostics quick reference . . . 14
H. When to escalate . . . 14
7. Best Practices — Software Asset Management (SAM) Program Standup & Governance 14
8. Deliverables Summary — Software Asset Management (SAM) Program Standup & Governance 15
Documentation . . . 15
Named program artifacts . . . 15
PowerShell scripts . . . 16
What the buyer receives . . . 16
Outcome . . . 16
Appendix A — PowerShell Scripts 17

Executive Summary — Software Asset Management (SAM) Program Standup & Governance

Most organizations "do SAM" as scattered spreadsheets and last-minute audit panic. There is no defined lifecycle, no named owners, no KPIs, and no repeatable process — so compliance is luck and savings are left on the table. This solution stands up a real Software Asset Management program: the SAM lifecycle, roles and RACI, SOPs and workflows, KPIs, and ITIL v4 alignment, so software is governed deliberately from request to retirement.

This is a complete, do-it-today package for standing up a SAM program, written for a SAM lead with no prior program-build experience. Every artifact is named once and reused everywhere, so you assign it once and repeat it across publishers, departments, and renewal cycles.

What you get

• A plain-language Solution Overview of the SAM lifecycle (request, entitlement, deployment, reconciliation, renewal/retirement), the roles and RACI, the tooling landscape, and ITIL v4 alignment.

• A Requirements Guide with verify / implement-if-missing for every prerequisite — an entitlement system of record, discovery data, a request/intake channel, named owners, and executive sponsorship.

• An Implementation Guide of precise, ordered steps: define scope and roles, stand up the system of record and intake, author the SOP set and workflows, set the KPI set and reporting, run the first governance cycle, then pilot, validate, and expand.

• Five PowerShell scripts for SAM-readiness discovery, process/data-coverage validation, a SAM KPI and maturity report, safe remediation of data gaps, and a SAM-scope group builder.

• Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

A deliberate, governed SAM program — a defined lifecycle, named owners with a clear RACI, a documented SOP set, a measured KPI set, and a single system of record — proven on a pilot scope before you scale it across the estate.

The deployment model

Assess, then Configure, then Pilot, then Validate, then Expand — with a pilot scope gate before any fleet-wide rollout.

Outcome

Software governed from request to retirement: known entitlements, reconciled deployment, owned renewals, and reportable compliance — replacing spreadsheet chaos and audit panic, validated on a pilot before you scale.

1. Solution Overview — Software Asset Management (SAM) Program Standup & Governance

What a SAM program is

Software Asset Management (SAM) is the discipline of governing software entitlements and deployments across their whole life. A SAM program is not a tool and not a spreadsheet — it is a defined lifecycle, a set of named owners, a documented set of standard operating procedures, a measured set of KPIs, and a single system of record, all working together so software is governed deliberately rather than rediscovered in an audit. This solution stands the program up in the repeatable way: named artifacts you assign once and reuse everywhere.

Why this matters

Spreadsheet SAM fails for predictable reasons. Nobody owns renewals, so contracts auto-renew at list price or lapse. Nobody reconciles entitlements against deployment, so you over-buy in one place and sit under-licensed (audit-exposed) in another. There are no KPIs, so "are we compliant?" has no answer until a publisher audit forces one. A real program fixes the cause, not the symptom: it assigns the lifecycle to owners, runs it on a cadence, and measures it — turning compliance and savings from luck into a repeatable outcome. The rest of this solution is the how; this section is the why.

The SAM lifecycle

The program governs every title through five stages. Each stage has an owner and a standard operating procedure (see the Implementation Guide).

1. Request — a user or team needs software; the request is captured through one intake channel, classified, and routed for approval against budget and policy.
1. Entitlement — an approved request is fulfilled from existing entitlements where possible (reharvested or pooled) or procured; the purchase and contract terms are recorded in the system of record.
1. Deployment — the entitled software is delivered to the user or device, and the assignment is recorded so deployment can later be reconciled against entitlement.
1. Reconciliation — on a cadence, deployed/used quantities are compared to entitled quantities to produce an Effective License Position (ELP): compliant, over-licensed (savings), or under-licensed (risk).
1. Renewal / Retirement — contracts approaching renewal are reviewed against the ELP and actual usage so you renew the right quantity, renegotiate, or retire and reharvest unused entitlements.

Roles and RACI

A program needs named people, not job titles in the abstract. The core roles are an Executive Sponsor (accountable for the program), a SAM Lead (runs the program day to day), a Procurement Liaison (owns

contracts and renewals), an IT/Deployment Owner (owns deployment and discovery data), and a Finance Liaison (owns budget and chargeback). The Implementation Guide names a single RACI — SAM RACI – Program – All Stages — that maps each lifecycle stage to these roles.

Standard operating procedures and workflows

Each lifecycle stage is documented as a repeatable SOP so the program runs the same way regardless of who is on shift. The Implementation Guide authors a named SOP set — SAM SOP Set – Program – All Stages — one procedure per stage, plus the supporting workflows (intake routing, approval, renewal review).

KPIs and the system of record

The program is measured by a named KPI set — SAM KPI Set – Program – All Stages — covering license compliance, reconciliation coverage, renewal lead time, reharvest savings, and request cycle time. All of it is anchored to a single system of record — the entitlement register — named SAM Register – Entitlements – System of Record, which holds entitlements, contracts, renewal dates, and the latest ELP.

ITIL v4 alignment

A SAM program is an ITIL v4 practice (IT Asset Management), and it interlocks with adjacent practices: Service Request Management feeds the Request stage, Change Enablement governs deployment changes, Service Configuration Management (the CMDB) shares asset data, and Supplier Management owns the publisher relationships behind renewals. Aligning to ITIL v4 means the program reuses the organization's existing request, change, and configuration practices instead of building parallel ones — and it makes the program legible to any ITSM-literate stakeholder.

What this is not

This program governs the process and the data. It does not replace a discovery tool, a procurement system, or a contract repository — it orchestrates them into one governed lifecycle with one system of record. Where tenant data is needed (for example, confirming entitlement and usage coverage), the included scripts read it through Microsoft Graph; they do not move money or change contracts.

The deployment model

> *Define scope and roles, stand up the system of record and intake, author the SOP set and KPI set, run the first governance cycle on a pilot scope, validate against the gate, then expand across the estate. The repeatable way: Assess, then Configure, then Pilot, then Validate, then Expand.*

2. Requirements Guide — Software Asset Management (SAM) Program Standup & Governance

Verify every requirement before standing up the program. Each entry: Description, Why, Verify, Implement-if-missing. Category-wide prerequisites are in `global-requirements-asset-mgt.md`.

2.1 A system of record for entitlements

Description: A single authoritative register of purchased entitlements, contracts, and renewal dates — named `SAM Register - Entitlements - System of Record`. A SAM/ITAM tool, a ServiceNow/CMDB instance, or, at minimum, a controlled spreadsheet/repository. Why: Every other stage (reconciliation, renewal, reporting) compares against this register; without one authoritative count there is no Effective License Position and no compliance answer. Verify: You can locate the current entitlement quantity and renewal date for a representative product in one place that everyone agrees is authoritative. Implement-if-missing: Stand up the register (start with the controlled spreadsheet if no tool exists), import current purchase orders and true-up statements, and declare it the single source of truth.

2.2 Deployment / usage discovery data

Description: A source of what is actually installed or used — Microsoft Intune, Configuration Manager, a discovery tool, or sign-in/usage telemetry. Why: Reconciliation compares deployed/used quantities to entitled quantities; without discovery data the ELP cannot be computed. Verify: You can export an installed-software or license-usage report for a sample of devices/users; run `SAMPROG-Discovery.ps1`. Implement-if-missing: Enable Intune/Configuration Manager inventory or onboard a discovery tool; for SaaS, enable usage/sign-in reporting.

2.3 A request / intake channel

Description: One channel through which all software requests enter the program — named `SAM Intake - Software Requests - Request Stage` — typically a ServiceNow request item or equivalent service-desk form. Why: A single intake is what makes the Request stage governable; scattered email/Teams requests can never be tracked, approved, or measured for cycle time. Verify: A test software request can be raised and is routed to the correct approver; run `SAMPROG-Validation.ps1` (checks intake-coverage signals). Implement-if-missing: Create the request item/form in the service-desk tool, define its fields (product, business justification, cost center), and publish it as the only supported way to ask for software.

2.4 Named owners and a RACI

Description: Named people for the core roles (Executive Sponsor, SAM Lead, Procurement Liaison, IT/Deployment Owner, Finance Liaison), mapped to the lifecycle in a single RACI — named `SAM RACI - Program - All Stages`. Why: A program is a set of repeatable decisions; decisions need accountable owners or they stall. Verify: For each lifecycle stage you can name the Responsible and Accountable person. Implement-if-missing: Assign the roles to named individuals, complete the RACI, and circulate it for acknowledgement.

2.5 Executive sponsorship

Description: A named Executive Sponsor with authority over IT, procurement, and finance stakeholders. Why: SAM crosses department lines (IT owns deployment, procurement owns contracts, finance owns budget); only a sponsor can resolve cross-team disputes and enforce the single intake. Verify: The sponsor is named in the RACI and has agreed to chair the governance cadence. Implement-if-missing: Secure a sponsor at the IT-director level or above and document the mandate.

2.6 A governance cadence and forum

Description: A recurring forum — named `SAM Governance Cadence - Program - Reconciliation & Renewal` — where the SAM Lead presents the KPI set and the ELP, and renewal decisions are made. Why: A lifecycle that is never reviewed drifts back to spreadsheet chaos; the cadence is what keeps it running. Verify: A recurring meeting exists with the right owners invited and an agenda anchored on the KPI set. Implement-if-missing: Schedule a monthly (reconciliation) and quarterly (renewal review) cadence; set the standing agenda from the KPI set.

2.7 A pilot scope

Description: A representative subset — one product family, one department, or one publisher — to prove the program before estate-wide rollout. Why: The program is validated on a pilot before broad adoption to confirm the lifecycle, data, and KPIs work end to end. Verify: A pilot scope and its data owners are identified; run `SAMPROG-Set-ScopeTargeting.ps1` to build the pilot scope group. Implement-if-missing: Choose a high-value, well-understood product/department as the first pass.

Checklist

- System of record exists — `SAM Register - Entitlements - System of Record`
- Discovery / usage data available — `SAMPROG-Discovery.ps1`
- Single intake channel — `SAM Intake - Software Requests - Request Stage`
- Named owners and RACI — `SAM RACI - Program - All Stages`
- Executive sponsor named
- Governance cadence scheduled — `SAM Governance Cadence - Program - Reconciliation & Renewal`
- Pilot scope identified — `SAMPROG-Set-ScopeTargeting.ps1`

3. Implementation Guide — Software Asset Management (SAM) Program Standup & Governance

Follow these steps in order. This guide stands up the program: scope and roles, the system of record and intake, the SOP set and KPI set, the first governance cycle, then pilot, validate, and expand. Artifact names are fixed — create them with the exact names shown and reuse them everywhere.

> *Model: define the program, prove it on a pilot scope, then expand. The repeatable way: Assess, then Configure, then Pilot, then Validate, then Expand.*

Step 1 — Assess current state

1. Run `SAMPROG-Discovery.ps1` to capture readiness signals: whether a discovery source and tenant

entitlement/usage data are reachable, and a snapshot of managed devices and assigned licenses.

1. Record the gaps against the Requirements Guide checklist. Do not proceed past a missing system of

record or missing discovery data — implement those first.

Step 2 — Define scope and roles

1. Confirm the program scope (which publishers/product families are in v1) and write it down.
2. Create the RACI named `SAM RACI - Program - All Stages`. Map each lifecycle stage (Request,

Entitlement, Deployment, Reconciliation, Renewal/Retirement) to Responsible, Accountable, Consulted, Informed against the core roles: Executive Sponsor, SAM Lead, Procurement Liaison, IT/Deployment Owner, Finance Liaison.

1. Circulate the RACI for acknowledgement so every owner has accepted their stage.

Step 3 — Stand up the system of record

1. Create or designate the register named `SAM Register - Entitlements - System of Record`.
2. Define its required fields: product, publisher, entitlement quantity, metric, contract ID,

renewal date, cost center, and latest Effective License Position (ELP).

1. Import current entitlements from purchase orders and true-up statements; declare this register the

single source of truth and retire competing spreadsheets.

Step 4 — Stand up the intake channel

1. Create the request item/form named `SAM Intake - Software Requests - Request Stage` in the

service-desk tool.

1. Define its fields (product, business justification, cost center, approver) and its routing so an

approved request reaches the Procurement Liaison.

1. Publish it as the only supported way to request software, and link it from the IT portal.

Step 5 — Author the SOP set

1. Create the SOP set named `SAM SOP Set - Program - All Stages` with one procedure per lifecycle

stage:

- Request SOP — capture, classify, and route through the intake; approve against budget/policy.
- Entitlement SOP — fulfil from reharvested/pooled entitlements first, then procure; record the

purchase and terms in the register.

- Deployment SOP — deliver to the user/device and record the assignment for later reconciliation.
- Reconciliation SOP — compare deployment/usage to entitlement; compute and record the ELP.

• Renewal/Retirement SOP — review upcoming renewals against ELP and usage; renew, renegotiate, or

retire and reharvest.

1. Store the SOP set alongside the register and reference it from the RACI.

Step 6 — Define the KPI set and reporting

1. Create the KPI set named `SAM KPI Set - Program - All Stages` with these measures and targets:

• License compliance (% of products with ELP at or above 100% entitled, none under-licensed).

• Reconciliation coverage (% of in-scope products reconciled within the cadence).

• Renewal lead time (days of notice before each renewal; target: review opened 90 days out).

• Reharvest savings (entitlements reclaimed and reused, in currency).

• Request cycle time (median days from intake to fulfilment).

1. Build the report with `SAMPROG-Report.ps1`, which renders the KPI set and a simple maturity

snapshot from available signals.

Step 7 — Establish the governance cadence

1. Schedule the forum named `SAM Governance Cadence - Program - Reconciliation & Renewal`: monthly for

reconciliation, quarterly for renewal review.

1. Set the standing agenda from the KPI set; the SAM Lead presents, the Executive Sponsor chairs,

renewal decisions are recorded back into the register.

Step 8 — Build the pilot scope

1. Run `SAMPROG-Set-ScopeTargeting.ps1` to create the pilot scope group (for example, one department

or one publisher) so the first governance cycle has a bounded, well-understood target.

1. Confirm the pilot scope's data owners are named in the RACI for that scope.

Step 9 — Run the first governance cycle on the pilot

1. Execute the lifecycle end to end on the pilot scope using the SOP set: process a real request,

record the entitlement, confirm deployment, then reconcile.

1. Run `SAMPROG-Validation.ps1` to confirm process/data coverage for the pilot scope (register

populated, discovery data present, intake live).

1. Run `SAMPROG-Report.ps1` to produce the first KPI/ELP snapshot for the pilot.

Step 10 — Validate against the pilot gate

1. Complete the Testing Guide. The pilot passes only when the lifecycle ran end to end, the ELP is computed for the pilot scope, and the KPI set reports without gaps.

1. Remediate any data gaps with `SAMPROG-Remediation.ps1` (runs with `-WhatIf` by default) before re-running validation.

Step 11 — Expand across the estate

1. After the pilot passes the gate, widen scope one publisher/department at a time, re-running `SAMPROG-Set-ScopeTargeting.ps1`, `SAMPROG-Validation.ps1`, and `SAMPROG-Report.ps1` for each new scope.

1. Keep the cadence running; the KPI set and ELP now cover the expanding estate.

Order of operations

```
Assess -> Roles/RACI -> System of record -> Intake -> SOP set -> KPI set ->
Cadence -> Pilot scope -> First cycle -> Validate gate -> Expand
```

5. Testing Guide — Software Asset Management (SAM) Program Standup & Governance

Run after the first governance cycle on the pilot scope (Implementation Step 9) and after each expansion. Each test confirms one part of the standing program works end to end.

Test 1 — System of record is authoritative

Do: Open `SAM Register - Entitlements - System of Record` and pick a representative pilot product. Expected: Entitlement quantity, contract ID, and renewal date are present and agreed as the single source of truth.

Test 2 — Intake routes a request

Do: Raise a test request through `SAM Intake - Software Requests - Request Stage`; run `SAMPROG-Validation.ps1`. Expected: The request is captured and routed to the approver/Procurement Liaison; validation reports the intake-coverage signal present.

Test 3 — Roles are owned

Do: For each lifecycle stage, name the Responsible and Accountable person from `SAM RACI - Program - All Stages`. Expected: Every stage has a named owner; no stage is unassigned.

Test 4 — Discovery data is available

Do: Run `SAMPROG-Discovery.ps1` for the pilot scope. Expected: A discovery/usage source is reachable and returns installed-software or license-usage data for the pilot sample.

Test 5 — Reconciliation computes an ELP

Do: Execute the Reconciliation SOP for the pilot scope, then run `SAMPROG-Report.ps1`. Expected: An Effective License Position (compliant / over / under) is computed and recorded in the register for each pilot product.

Test 6 — KPI set reports

Do: Run `SAMPROG-Report.ps1`. Expected: License compliance, reconciliation coverage, renewal lead time, reharvest savings, and request cycle time render for the pilot scope without gaps.

Test 7 — Governance cadence runs

Do: Hold the first `SAM Governance Cadence - Program - Reconciliation & Renewal` session. Expected: The SAM Lead presents the KPI set/ELP, the sponsor chairs, and at least one renewal decision is recorded back into the register.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1-6 pass; the lifecycle ran end to end and an ELP is computed for the pilot scope
Expand pass	Tests 1-6 pass for each new scope and Test 7 (cadence) is running before estate-wide rollout

6. Troubleshooting Guide — Software Asset Management (SAM) Program Standup & Governance

Use `SAMPROG-Discovery.ps1` and `SAMPROG-Validation.ps1` for readiness and coverage state, `SAMPROG-Report.ps1` for the KPI/ELP view, and `SAMPROG-Remediation.ps1` (runs with `-WhatIf` by default) for safe fixes to data gaps.

A. Reconciliation cannot compute an ELP

Symptom	Cause	Remediation
ELP blank for a product	Entitlement missing from the register	Add the entitlement to `SAM Register - Entitlements - System of Record`; re-run `SAMPROG-Report.ps1`

Symptom	Cause	Remediation
ELP blank for a product	No deployment/usage data	Run `SAMPROG-Discovery.ps1`; enable Intune/discovery or SaaS usage reporting; re-run validation
ELP looks wrong	Stale or duplicate register rows	Run `SAMPROG-Remediation.ps1` to flag duplicates/gaps; clean the register and reconcile again

B. Requests bypass the program

- Side-channel requests (email/Teams) are invisible to approval and metrics. Enforce

`SAM Intake - Software Requests - Request Stage` as the only supported channel and route all others back through it; have the Executive Sponsor reinforce the mandate.

C. A lifecycle stage stalls

Symptom	Cause	Remediation
Requests pile up unapproved	No Accountable owner for the stage	Complete `SAM RACI - Program - All Stages`; name an Accountable person per stage
Renewals slip	Renewal review opened too late	Open the renewal review 90 days out per the Renewal/Retirement SOP; surface upcoming dates in `SAMPROG-Report.ps1`

D. KPI report has gaps

- Missing measures usually mean a missing data source. Run `SAMPROG-Validation.ps1` to see which of

register / discovery / intake coverage is absent, then implement the missing prerequisite from the Requirements Guide and re-run `SAMPROG-Report.ps1`.

E. Graph scripts fail to connect

- `SAMPROG-Discovery.ps1`, `SAMPROG-Report.ps1`, and `SAMPROG-Set-ScopeTargeting.ps1` use Microsoft

Graph. If connection fails, confirm the Microsoft Graph PowerShell SDK is installed and the signed-in account has the read scopes the script requests (consent on first run). Re-run after consent.

F. Program drifts back to spreadsheets

- The cause is almost always a missing cadence. Confirm `SAM Governance Cadence - Program -

Reconciliation & Renewal` is scheduled and chaired by the sponsor; a lifecycle that is never reviewed always decays.

G. Diagnostics quick reference

```
.\SAMPROG-Discovery.ps1                 # readiness: discovery source, tenant coverage, devices/licenses
.\SAMPROG-Validation.ps1 -Scope Pilot   # process/data coverage for a scope
.\SAMPROG-Report.ps1                     # KPI set + ELP snapshot
.\SAMPROG-Remediation.ps1               # -WhatIf by default; add -Apply to act
```

H. When to escalate

If the register is populated, discovery data is present, the intake is live, and the KPI set still reports gaps after remediation, capture the `SAMPROG-Validation.ps1` and `SAMPROG-Report.ps1` output and review the data sources with the IT/Deployment Owner and Procurement Liaison at the governance cadence.

7. Best Practices — Software Asset Management (SAM) Program Standup & Governance

- One system of record, always. Make `SAM Register - Entitlements - System of Record` the single

source of truth and retire competing spreadsheets — split records are how compliance answers go stale.

- One intake, no exceptions. Force every request through `SAM Intake - Software Requests - Request

Stage`; a side channel that bypasses intake is invisible to approval, budget, and cycle-time metrics.

- Name an accountable owner per stage. Keep `SAM RACI - Program - All Stages` current; a lifecycle

stage with no Accountable person stalls — this is the most common reason programs revert to spreadsheet chaos.

- Reharvest before you buy. The Entitlement SOP fulfils from reclaimed/pooled entitlements first;

reharvesting unused licenses is usually the largest, fastest saving in the program.

- Reconcile on a cadence, not on audit. Run reconciliation monthly so the Effective License Position

is always current; never let a publisher audit be the first time you compute it.

- Open renewals early. Target a renewal review 90 days out so you renew the right quantity against

real usage rather than auto-renewing at list price.

- Measure the program, not just the licenses. Track the full `SAM KPI Set - Program - All Stages`

(compliance, coverage, lead time, savings, cycle time); a downward trend in savings or coverage is an early warning.

- Align to ITIL v4, do not duplicate it. Reuse the organization's existing request, change, and

configuration practices rather than building parallel ones; it keeps the program legible and lower-maintenance.

• Prove it on a pilot first. Run the whole lifecycle on one publisher/department before estate-wide rollout — the pilot gate catches data and ownership gaps cheaply.

• Document the cadence and decisions. Record every renewal/retirement decision back into the register so the program is auditable and repeatable across cycles and owners.

8. Deliverables Summary — Software Asset Management (SAM) Program Standup & Governance

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What a SAM program is, the lifecycle, roles/RACI, KPIs, system of record, and ITIL v4 alignment (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Asset Mgt)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps to stand up and run the program
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

Named program artifacts

Artifact	Name
Entitlement register	`SAM Register - Entitlements - System of Record`
Request intake	`SAM Intake - Software Requests - Request Stage`
RACI	`SAM RACI - Program - All Stages`
SOP set	`SAM SOP Set - Program - All Stages`
KPI set	`SAM KPI Set - Program - All Stages`
Governance forum	`SAM Governance Cadence - Program - Reconciliation & Renewal`

PowerShell scripts

Script	What it does
`SAMPROG-Discovery.ps1`	Reports SAM-readiness signals: discovery source, tenant entitlement/usage coverage, managed devices and assigned licenses (Microsoft Graph, read-only)
`SAMPROG-Validation.ps1`	Confirms process/data coverage for a scope: register populated, discovery data present, intake live (read-only)
`SAMPROG-Report.ps1`	Renders the KPI set and a maturity snapshot, plus an Effective License Position view (Microsoft Graph, read-only)
`SAMPROG-Remediation.ps1`	Safe remediation of data gaps; runs with `-WhatIf` by default
`SAMPROG-Set-ScopeTargeting.ps1`	Builds the SAM pilot/expansion scope group (Microsoft Graph)

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable program following the model: Assess, then Configure, then Pilot, then Validate, then Expand.

Outcome

A deliberate, governed SAM program — defined lifecycle, named owners, documented SOPs, measured KPIs, and a single system of record — replacing spreadsheet chaos and audit panic, proven on a pilot scope before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SAMPROG-Discovery.ps1

Purpose: Reports SAM-program readiness signals so you can assess current state before standing up the program: whether a tenant entitlement source (assigned licenses) and a deployment/usage discovery source (Intune managed devices) are reachable, and a high-level count of managed Windows devices and subscribed SKUs. Read-only. This supports the Assess step — it does not change any tenant data. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SAMPROG-Discovery.ps1 Requires the Microsoft Graph PowerShell SDK. On first run, consent to the read scopes: Organization.Read.All, DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A readiness summary: subscribed SKU count (entitlement source), managed Windows device count (discovery source), and a PASS/GAP line per signal you can record against the Requirements checklist. .NOTES Asset Mgt — SAM Program. Read-only against Microsoft Graph. The repeatable way. #> [CmdletBinding()] param()

How to run: powershell -ExecutionPolicy Bypass -File .\SAMPROG-Discovery.ps1 Requires the Microsoft Graph PowerShell SDK. On first run, consent to the read scopes: Organization.Read.All, DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A readiness summary: subscribed SKU count (entitlement source), managed Windows device count (discovery source), and a PASS/GAP line per signal you can record against the Requirements checklist. .NOTES Asset Mgt — SAM Program. Read-only against Microsoft Graph. The repeatable way. #> [CmdletBinding()] param()

SAMPROG-Remediation.ps1

Purpose: Cleans common data-quality gaps that block reconciliation, working only on a local export of `SAM Register - Entitlements - System of Record` (a CSV) — never on tenant data, contracts, or money. It flags duplicate product/SKU rows, rows missing an entitled quantity, and rows missing a renewal date, and (when applied) writes a normalized, de-duplicated copy you can re-import. Runs with -WhatIf by default, so it only reports unless you explicitly apply. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\SAMPROG-Remediation.ps1 ` -RegisterPath C:\SAM\entitlement-register.csv Apply: ...\SAMPROG-Remediation.ps1 -RegisterPath C:\SAM\entitlement-register.csv ` -OutputPath C:\SAM\entitlement-register.clean.csv -WhatIf:$false EXPECTED RESULTS In preview: a list of duplicates and missing-field rows and what would change. When applied: a cleaned CSV at -OutputPath (duplicates collapsed, blanks flagged), leaving the original untouched. .NOTES Asset Mgt — SAM Program. Local-file only; -WhatIf by default. The repeatable way. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([Parameter(Mandatory)][string]$RegisterPath, [string]$OutputPath)

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\SAMPROG-Remediation.ps1 `-RegisterPath C:\SAM\entitlement-register.csv` Apply: `...\SAMPROG-Remediation.ps1 -RegisterPath C:\SAM\entitlement-register.csv` -OutputPath C:\SAM\entitlement-register.clean.csv -WhatIf:$false EXPECTED RESULTS In preview: a list of duplicates and missing-field rows and what would change. When applied: a cleaned CSV at -OutputPath (duplicates collapsed, blanks flagged), leaving the original untouched. .NOTES Asset Mgt — SAM Program. Local-file only; -WhatIf by default. The repeatable way. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([Parameter(Mandatory)][string]$RegisterPath, [string]$OutputPath)

SAMPROG-Report.ps1

Purpose: Produces the standing program report. It computes an Effective License Position per product by comparing entitled quantities (from an export of the entitlement register) against assigned/used quantities pulled from Microsoft Graph (subscribed SKUs), and renders the `SAM KPI Set - Program - All Stages` measures it can derive plus a simple maturity snapshot. Read-only against Graph. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SAMPROG-Report.ps1 ` -RegisterPath C:\SAM\entitlement-register.csv -CsvPath C:\Temp\sam-elp.csv The register CSV should include columns: Product, SkuPartNumber, Entitled, RenewalDate. Requires the Microsoft Graph SDK and Organization.Read.All (subscribedSkus). EXPECTED RESULTS A per-product ELP table (Entitled vs Assigned -> Compliant/Over/Under), the derivable KPI measures (compliance %, renewal lead-time flags), and an optional CSV export. No tenant data is modified. .NOTES Asset Mgt — SAM Program. Read-only against Microsoft Graph. The repeatable way. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$RegisterPath, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\SAMPROG-Report.ps1 ` -RegisterPath C:\SAM\entitlement-register.csv -CsvPath C:\Temp\sam-elp.csv The register CSV should include columns: Product, SkuPartNumber, Entitled, RenewalDate. Requires the Microsoft Graph SDK and Organization.Read.All (subscribedSkus). EXPECTED RESULTS A per-product ELP table (Entitled vs Assigned -> Compliant/Over/Under), the derivable KPI measures (compliance %, renewal lead-time flags), and an optional CSV export. No tenant data is modified. .NOTES Asset Mgt — SAM Program. Read-only against Microsoft Graph. The repeatable way. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$RegisterPath, [string]$CsvPath)

SAMPROG-Set-ScopeTargeting.ps1

Purpose: Creates (or reuses) the Entra security group that defines a SAM program scope — the pilot first, then each expansion — so the first governance cycle and every later wave run against a bounded, well-understood target. Optionally seeds members from a CSV of user UPNs or device names. This is the only included script that writes to the tenant; it creates a group and adds members, nothing else (no licenses changed, no money moved). HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\SAMPROG-Set-ScopeTargeting.ps1 ` -GroupName "SAM Scope - Pilot - Finance" -MembersCsv C:\SAM\pilot-users.csv Apply: ...\SAMPROG-Set-ScopeTargeting.ps1 -GroupName "SAM Scope - Pilot - Finance" ` -MembersCsv C:\SAM\pilot-users.csv -WhatIf:$false MembersCsv: a CSV with a 'Upn' column (users). Requires the Microsoft Graph SDK and Group.ReadWrite.All (and User.Read.All to resolve UPNs). EXPECTED RESULTS In preview: the group that would be created/reused and the members that would be added. When applied: the security group exists and contains the listed members, ready to target with the first governance cycle. .NOTES Asset Mgt — SAM Program. Writes to Microsoft Entra; -WhatIf by default. The repeatable way. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$GroupName, [string]$MembersCsv)

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\SAMPROG-Set-ScopeTargeting.ps1 `-GroupName "SAM Scope - Pilot - Finance" -MembersCsv C:\SAM\pilot-users.csv Apply: ...\SAMPROG-Set-ScopeTargeting.ps1 -GroupName "SAM Scope - Pilot - Finance"` -MembersCsv C:\SAM\pilot-users.csv -WhatIf:$false MembersCsv: a CSV with a 'Upn' column (users). Requires the Microsoft Graph SDK and Group.ReadWrite.All (and User.Read.All to resolve UPNs). EXPECTED RESULTS In preview: the group that would be created/reused and the members that would be added. When applied: the security group exists and contains the listed members, ready to target with the first governance cycle. .NOTES Asset Mgt

— SAM Program. Writes to Microsoft Entra; -WhatIf by default. The repeatable way. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$GroupName, [string]$MembersCsv)

SAMPROG-Validation.ps1

Purpose: Validates that the program's foundations are in place for a given scope before (and after) the first governance cycle: the entitlement register is populated, a discovery/usage source is readable, and the intake channel is recorded as live. This is a process gate — it checks coverage signals; it is read-only and changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SAMPROG-Validation.ps1 -Scope Pilot ` -RegisterPath C:\SAM\entitlement-register.csv -IntakeLive -RegisterPath points at an export of `SAM Register - Entitlements - System of Record` (CSV). -IntakeLive asserts the `SAM Intake - Software Requests - Request Stage` channel is published. Requires the Microsoft Graph SDK for the discovery check (DeviceManagementManagedDevices.Read.All). EXPECTED RESULTS "COVERAGE COMPLETE" when the register has rows, discovery data is readable, and intake is marked live; otherwise a per-signal GAP list and a non-zero exit code. .NOTES Asset Mgt — SAM Program. Read-only. The repeatable way. #> [CmdletBinding()] param([string]$Scope = 'Pilot', [string]$RegisterPath, [switch]$IntakeLive)

How to run: powershell -ExecutionPolicy Bypass -File .\SAMPROG-Validation.ps1 -Scope Pilot `-RegisterPath C:\SAM\entitlement-register.csv -IntakeLive -RegisterPath points at an export of` SAM Register - Entitlements - System of Record `(CSV). -IntakeLive asserts the` SAM Intake - Software Requests - Request Stage` channel is published. Requires the Microsoft Graph SDK for the discovery check (DeviceManagementManagedDevices.Read.All). EXPECTED RESULTS "COVERAGE COMPLETE" when the register has rows, discovery data is readable, and intake is marked live; otherwise a per-signal GAP list and a non-zero exit code. .NOTES Asset Mgt — SAM Program. Read-only. The repeatable way. #> [CmdletBinding()] param([string]$Scope = 'Pilot', [string]$RegisterPath, [switch]$IntakeLive)